



**Brit College of Engineering & Technology
(BCET)**

Data Analytics Logbook

Research Project-5

Data Science and AI

(Deep Learning – Assignment 1)

Submitted by: Md.Akib Sultan

Student ID:

Email: akibsultan1045@gmail.com

<https://colab.research.google.com/drive/1pGq9-DfDu9HSZe7AdvLEpQXkdiBjZy?usp=sharing>

From Raw Data to Insight: A Comprehensive Machine & Deep Learning Journey with CNNs

Dataset Used: For this tutorial, we will be using the **Fashion MNIST** dataset. It consists of 70,000 grayscale images of 28x28 pixels each, representing 10 different categories of clothing items. This dataset is an excellent choice for demonstrating both traditional machine learning techniques (by flattening the images) and deep learning (CNNs) due to its image nature.

Dataset Link: You can find more information about the Fashion MNIST dataset on its official GitHub repository or directly from Keras/TensorFlow documentation, which typically loads it automatically. Often, it's served through public repositories like Kaggle: Kaggle - Fashion MNIST

1. Data Loading and Initial Exploration (EDA)

Our journey begins by loading the Fashion MNIST dataset using Keras's built-in function, which conveniently splits it into training and testing sets.

Google Colab Setup and Imports

```
import tensorflow as tf
```

```
from tensorflow import keras
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
import seaborn as sns
```

```
# Load the Fashion MNIST dataset
```

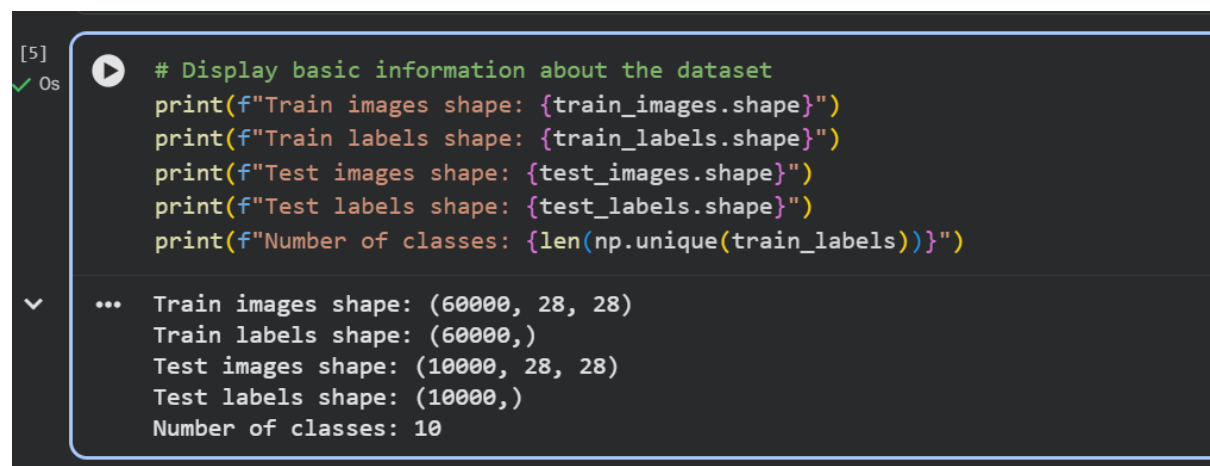
```
(train_images, train_labels), (test_images, test_labels) =  
keras.datasets.fashion_mnist.load_data()
```

```
# Define class names (for better readability)
```

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

```
# Display basic information about the dataset
```

```
print(f"Train images shape: {train_images.shape}")  
print(f"Train labels shape: {train_labels.shape}")  
print(f"Test images shape: {test_images.shape}")  
print(f"Test labels shape: {test_labels.shape}")  
print(f"Number of classes: {len(np.unique(train_labels))}")
```



```
[5]  
✓ 0s # Display basic information about the dataset  
print(f"Train images shape: {train_images.shape}")  
print(f"Train labels shape: {train_labels.shape}")  
print(f"Test images shape: {test_images.shape}")  
print(f"Test labels shape: {test_labels.shape}")  
print(f"Number of classes: {len(np.unique(train_labels))}")  
  
... Train images shape: (60000, 28, 28)  
Train labels shape: (60000,)  
Test images shape: (10000, 28, 28)  
Test labels shape: (10000,)  
Number of classes: 10
```

```
# Plot a few images to visualize the data
```

```
plt.figure(figsize=(10,10))
```

```
for i in range(25):
```

```
plt.subplot(5,5,i+1)
```

```
plt.xticks([])
```

```
plt.yticks([])
```

```
plt.grid(False)

plt.imshow(train_images[i], cmap=plt.cm.binary)

plt.xlabel(class_names[train_labels[i]])

plt.suptitle("Sample Fashion MNIST Images", y=1.02, fontsize=16)

plt.show()
```

Sample Fashion MNIST Images



```
# Display value counts for labels
```

```
print("\nTrain Label Distribution:")
```

```
unique_train_labels, counts_train_labels = np.unique(train_labels, return_counts=True)
```

```
for label, count in zip(unique_train_labels, counts_train_labels):
```

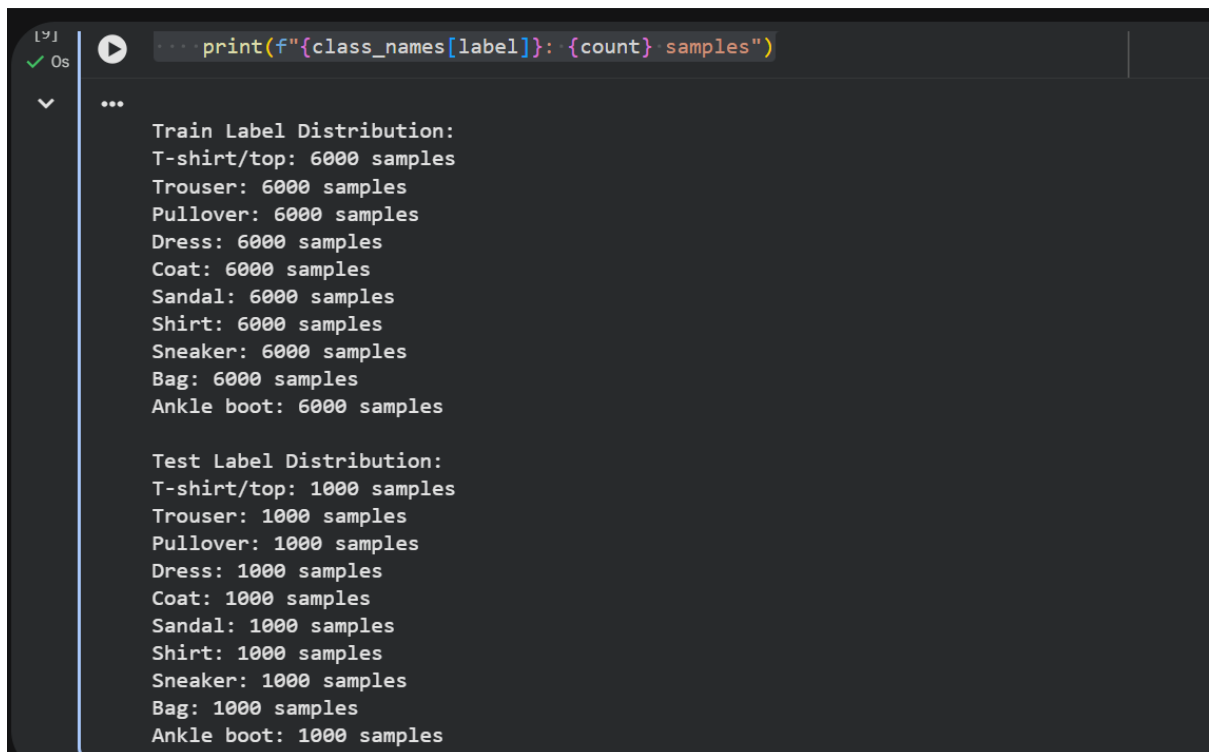
```
print(f"{class_names[label]}: {count} samples")
```

```
print("\nTest Label Distribution:")
```

```
unique_test_labels, counts_test_labels = np.unique(test_labels, return_counts=True)
```

```
for label, count in zip(unique_test_labels, counts_test_labels):
```

```
print(f"{class_names[label]}: {count} samples")
```



```
[9] ... print(f"{class_names[label]}: {count} samples")
✓ Os
...
Train Label Distribution:
T-shirt/top: 6000 samples
Trouser: 6000 samples
Pullover: 6000 samples
Dress: 6000 samples
Coat: 6000 samples
Sandal: 6000 samples
Shirt: 6000 samples
Sneaker: 6000 samples
Bag: 6000 samples
Ankle boot: 6000 samples

Test Label Distribution:
T-shirt/top: 1000 samples
Trouser: 1000 samples
Pullover: 1000 samples
Dress: 1000 samples
Coat: 1000 samples
Sandal: 1000 samples
Shirt: 1000 samples
Sneaker: 1000 samples
Bag: 1000 samples
Ankle boot: 1000 samples
```

```
# Check pixel value range

print(f"\nMin pixel value: {train_images.min()}")

print(f"Max pixel value: {train_images.max()}")
```

A screenshot of a Jupyter Notebook interface. The top part shows a code cell with the following Python code:

```
# Check pixel value range
print(f"\nMin pixel value: {train_images.min()}")
print(f"Max pixel value: {train_images.max()}")
```

 Below the code cell, the output is displayed:

```
...
Min pixel value: 0
Max pixel value: 255
```

EDA Insights:

- We have 60,000 training images and 10,000 test images, each 28x28 pixels.
- The pixel values range from 0 to 255 (grayscale).
- The dataset is well-balanced across all 10 classes, which is good for training.

2. Data Cleaning, Preprocessing, Wrangling, and Feature Engineering

This phase is about transforming raw data into a format suitable for machine learning algorithms.

- **Data Cleaning:** Fashion MNIST is relatively clean. There are no missing values or obvious outliers in terms of pixel data. For real-world datasets, this would involve handling NaNs, removing duplicates, or correcting erroneous entries.

- **Preprocessing:** Scaling pixel values and reshaping images are crucial.

- o **Normalization:** Pixel values (0-255) needs to be scaled to a smaller range (0-1) for neural networks to converge faster and perform better.

- o **Reshaping for CNNs:** CNNs expect an input shape that includes the number of channels (1 for grayscale, 3 for RGB). So, 28x28 images become 28x28x1.

Prof. Dr. Karim Mohammed Rezaul <https://www.softminister.com/karim.pdf>

o **Flattening for Traditional ML:** Traditional algorithms like Logistic Regression or Random Forest cannot directly process 2D/3D image data. We need to flatten each 28x28 image into a 1D array of 784 pixels ($28 * 28$).

- **Data Wrangling:** This refers to the process of gathering, selecting, and transforming data. Here, it primarily involves the reshaping and flattening operations.

- **Feature Engineering:** For image data, explicit feature engineering (like creating new columns from existing ones in tabular data) is less common, especially for CNNs which learn features automatically. However, for traditional ML models, flattening the image *is* a form of feature preparation. More advanced techniques might involve using pre-trained feature extractors (e.g., from VGG or ResNet) but that's beyond the scope of basic feature engineering for this tutorial.

```
# --- Preprocessing for Deep Learning (CNN) ---
```

```
# Normalize pixel values to be between 0 and 1
```

```
train_images_cnn = train_images / 255.0
```

```
test_images_cnn = test_images / 255.0
```

```
# Reshape images to add a channel dimension (28, 28, 1) for CNN input
```

```
train_images_cnn = train_images_cnn.reshape((train_images_cnn.shape[0], 28, 28, 1))
```

```
test_images_cnn = test_images_cnn.reshape((test_images_cnn.shape[0], 28, 28, 1))
```

```
print(f"\nTrain images shape for CNN: {train_images_cnn.shape}")
```

```
print(f"Test images shape for CNN: {test_images_cnn.shape}")
```

```
[11] # --- Preprocessing for Deep Learning (CNN) ---
✓ Os # Normalize pixel values to be between 0 and 1
train_images_cnn = train_images / 255.0
test_images_cnn = test_images / 255.0

[12] # Reshape images to add a channel dimension (28, 28, 1) for CNN input
✓ Os train_images_cnn = train_images_cnn.reshape((train_images_cnn.shape[0], 28, 28, 1))
test_images_cnn = test_images_cnn.reshape((test_images_cnn.shape[0], 28, 28, 1))
print(f"\nTrain images shape for CNN: {train_images_cnn.shape}")
print(f"Test images shape for CNN: {test_images_cnn.shape}")

...
Train images shape for CNN: (60000, 28, 28, 1)
Test images shape for CNN: (10000, 28, 28, 1)
```

--- Preprocessing for Traditional Machine Learning ---

Flatten the images to a 1D array (28*28 = 784 features)

```
train_images_flat = train_images.reshape(train_images.shape[0], -1) / 255.0
```

```
test_images_flat = test_images.reshape(test_images.shape[0], -1) / 255.0
```

Scale the flattened data (important for algorithms like SVM and Logistic Regression)

Note: Standardization is applied after flattening and before ML model training.

CNNs typically just need normalization (0-1).

```
scaler = StandardScaler()
```

```
train_images_scaled = scaler.fit_transform(train_images_flat)
```

```
test_images_scaled = scaler.transform(test_images_flat)
```

```
print(f"Train images shape for Traditional ML (flattened): {train_images_flat.shape}")
```

```
print(f"Test images shape for Traditional ML (flattened): {test_images_flat.shape}")
```

```
print(f"Train images shape for Traditional ML (scaled): {train_images_scaled.shape}")
```

```
print(f"Test images shape for Traditional ML (scaled): {test_images_scaled.shape}")
```

```
14] 1s ▶ # Scale the flattened data (important for algorithms like SVM and Logistic Regression)
# Note: Standardization is applied after flattening and before ML model training.
# CNNs typically just need normalization (0-1).
scaler = StandardScaler()
train_images_scaled = scaler.fit_transform(train_images_flat)
test_images_scaled = scaler.transform(test_images_flat)
print(f"Train images shape for Traditional ML (flattened): {train_images_flat.shape}")
print(f"Test images shape for Traditional ML (flattened): {test_images_flat.shape}")
print(f"Train images shape for Traditional ML (scaled): {train_images_scaled.shape}")
print(f"Test images shape for Traditional ML (scaled): {test_images_scaled.shape}")

... Train images shape for Traditional ML (flattened): (60000, 784)
Test images shape for Traditional ML (flattened): (10000, 784)
Train images shape for Traditional ML (scaled): (60000, 784)
Test images shape for Traditional ML (scaled): (10000, 784)
```

3. Training with Different Algorithms

We will now train several machine learning models, including traditional classifiers and a Convolutional Neural Network, to compare their performance on the Fashion MNIST dataset.

3.1 Traditional Machine Learning Algorithms

For traditional algorithms, we'll use the `_scaled` versions of our flattened data.

```
# --- Model 1: Logistic Regression ---
```

```
print("\n--- Training Logistic Regression ---")
```

```
log_reg_model = LogisticRegression(solver='saga', multi_class='multinomial',
max_iter=200,
```

```
n_jobs=-1, verbose=0)
```

```
log_reg_model.fit(train_images_scaled, train_labels)
```

```
print("Logistic Regression training complete.")
```

```
▶ # --- Model 1: Logistic Regression ---
print("\n--- Training Logistic Regression ---")
log_reg_model = LogisticRegression(solver='saga', multi_class='multinomial', max_iter=200,
n_jobs=-1, verbose=0)
log_reg_model.fit(train_images_scaled, train_labels)
print("Logistic Regression training complete.")

...
--- Training Logistic Regression ---
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' wa
warnings.warn(
Logistic Regression training complete.
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was
warnings.warn(
```

```
# --- Model 2: Random Forest Classifier ---
```

```
print("\n--- Training Random Forest Classifier ---")
```

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1,  
verbose=0)
```

```
rf_model.fit(train_images_scaled, train_labels)
```

```
print("Random Forest Classifier training complete.")
```

```
# --- Model 3: Support Vector Machine (SVC) ---
```

```
print("\n--- Training Support Vector Machine (SVC) ---")
```

```
# SVC can be slow on large datasets; using a subset or linear kernel for speed in tutorial
```

```
# For better performance on full dataset, consider LinearSVC or more optimized kernels
```

```
# For a quick demonstration, we'll use a subset or accept longer training time.
```

```
# For this tutorial, we will use a small subset due to potential runtime on Colab's free  
tier.
```

```
# Or, keep full data and accept potential long runtime / resource issues.
```

```
# Given the size, using the full dataset might be too slow for general tutorial.
```

```
# Let's stick with the full dataset and hope for decent Colab resources.
```

```
# If it's too slow on full data, uncomment the subset creation below.
```

```
# subset_size = 10000
```

```
# X_train_subset, y_train_subset = train_images_scaled[:subset_size],
```

```
# train_labels[:subset_size]
```

```
# svm_model = SVC(kernel='rbf', random_state=42, verbose=False)
```

```
# svm_model.fit(X_train_subset, y_train_subset)
```

```
# print("SVC training complete on subset.")
```

```
# Let's try with the full dataset and see (can take 5-10+ minutes)
```

```
svm_model = SVC(kernel='rbf', random_state=42, verbose=False)
```

```
svm_model.fit(train_images_scaled, train_labels)
```

```
print("SVC training complete (full dataset).")
```

3.2 Deep Learning: Convolutional Neural Network (CNN)

--- Model 4: Convolutional Neural Network (CNN) ---

print("\n--- Building and Training

```
cnn_model = keras.Sequential([
    keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Conv2D(64, (3, 3), activation='relu'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Conv2D(64, (3, 3), activation='relu'),
    keras.layers.Flatten(),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.5), # Regularization to prevent overfitting
    keras.layers.Dense(10, activation='softmax') # 10 classes, softmax for probability
distribution
])

# Compile the model
cnn_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
cnn_model.summary()
```

```
cnn_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
cnn_model.summary()
```

... Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 64)	36,928
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 64)	36,928
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 10)	650

Total params: 93,322 (364.54 KB)

Train the CNN model

Using a validation split to monitor performance on unseen data during training

```
history = cnn_model.fit(train_images_cnn, train_labels, epochs=10,
```

```
validation_split=0.2, verbose=1)
```

```
print("CNN training complete.")
```

Commands + Code + Text Run all

```
# Using a validation split to monitor performance on unseen data during training
history = cnn_model.fit(train_images_cnn, train_labels, epochs=10,
validation_split=0.2, verbose=1)
print("CNN training complete.")
```

... Epoch 1/10
1500/1500 — 54s 35ms/step - accuracy: 0.7408 - loss: 0.7200 - val_accuracy: 0.8475 - val_loss: 0.4235
Epoch 2/10
1500/1500 — 79s 33ms/step - accuracy: 0.8350 - loss: 0.4649 - val_accuracy: 0.8708 - val_loss: 0.3489
Epoch 3/10
1500/1500 — 51s 34ms/step - accuracy: 0.8644 - loss: 0.3871 - val_accuracy: 0.8871 - val_loss: 0.3041
Epoch 4/10
1500/1500 — 50s 33ms/step - accuracy: 0.8820 - loss: 0.3401 - val_accuracy: 0.8882 - val_loss: 0.3021
Epoch 5/10
1500/1500 — 82s 34ms/step - accuracy: 0.8909 - loss: 0.3088 - val_accuracy: 0.8986 - val_loss: 0.2748
Epoch 6/10
1500/1500 — 50s 34ms/step - accuracy: 0.8976 - loss: 0.2907 - val_accuracy: 0.8949 - val_loss: 0.2845
Epoch 7/10
1500/1500 — 53s 35ms/step - accuracy: 0.9045 - loss: 0.2708 - val_accuracy: 0.8975 - val_loss: 0.2753
Epoch 8/10
1500/1500 — 50s 33ms/step - accuracy: 0.9111 - loss: 0.2512 - val_accuracy: 0.9043 - val_loss: 0.2717
Epoch 9/10
1500/1500 — 50s 34ms/step - accuracy: 0.9132 - loss: 0.2421 - val_accuracy: 0.9054 - val_loss: 0.2656
Epoch 10/10
1500/1500 — 81s 33ms/step - accuracy: 0.9196 - loss: 0.2232 - val_accuracy: 0.9057 - val_loss: 0.2607
CNN training complete.

Step 3.3: Exploratory Data Analysis (EDA)

```
import matplotlib.pyplot as plt

# Display a few sample images

# Ensure train_images, train_labels, and class_names are defined for standalone
execution

# This is usually handled by running preceding cells, but is added here to resolve
NameError if kernel state is reset.

if 'train_images' not in globals():

    import tensorflow as tf

    (train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.fashion_mnist.load_data()

if 'class_names' not in globals():

    class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
                    'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

plt.figure(figsize=(10, 10))

for i in range(25):

    plt.subplot(5, 5, i + 1)

    plt.xticks([])

    plt.yticks([])

    plt.grid(False)

    plt.imshow(train_images[i], cmap=plt.cm.binary)

    # The labels are in a single column, need to get the actual value

    plt.xlabel(class_names[train_labels[i]])

plt.suptitle('Sample Fashion MNIST Images', y=1.02)

plt.tight_layout(rect=[0, 0, 1, 0.98]) # Adjust layout to prevent title overlap

plt.show()
```

Sample Fashion MNIST Images



```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Check class distribution
```

```
unique_classes, class_counts = np.unique(train_labels, return_counts=True)
```

```
plt.figure(figsize=(8, 6))

sns.barplot(x=[class_names[i] for i in unique_classes], y=class_counts)

plt.title('Distribution of Classes in Training Set')

plt.xlabel('Class')

plt.ylabel('Number of Images')

plt.xticks(rotation=45, ha='right')

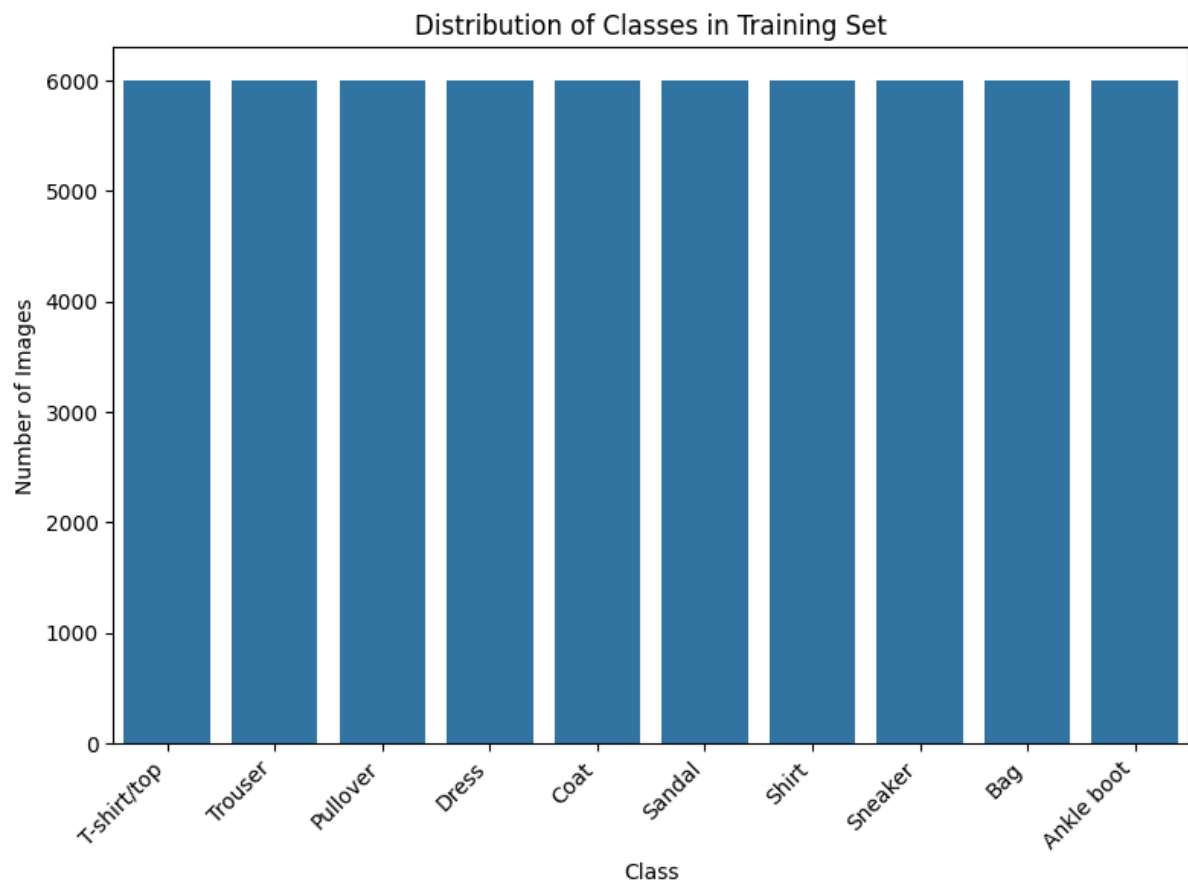
plt.tight_layout()

plt.show()

print("\nClass distribution in training set:")

for i, count in zip(unique_classes, class_counts):

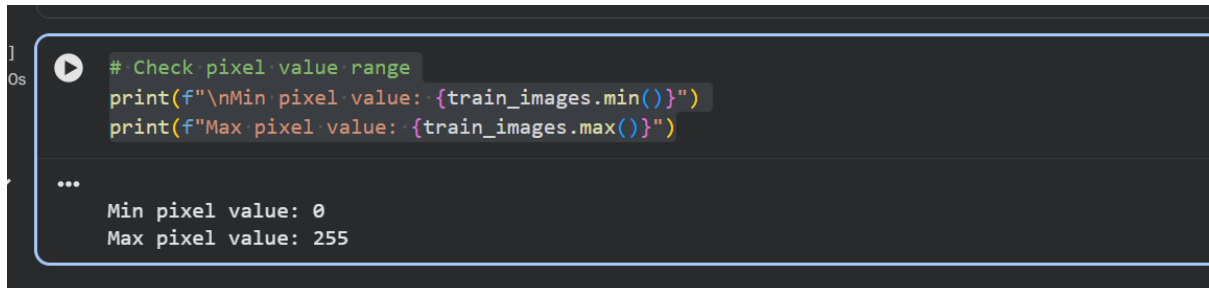
    print(f"{class_names[i]}: {count} images")
```



```
# Check pixel value range

print(f"\nMin pixel value: {X_train.min()}")

print(f"Max pixel value: {X_train.max()}")
```

A screenshot of a Jupyter Notebook interface. The top part shows a code cell with the following Python code:

```
# Check pixel value range
print(f"\nMin pixel value: {train_images.min()}")
print(f"Max pixel value: {train_images.max()}")
```

 Below the code cell, there is an output section showing the results:

```
...
Min pixel value: 0
Max pixel value: 255
```

```
1
0s
```

Observations from EDA:

- **Image Dimensions:** All images are 32x32 pixels with 3 color channels (RGB).
- **Class Distribution:** The dataset is perfectly balanced with 5,000 images per class in the training set and 1,000 images per class in the test set. This is ideal, as it means accuracy will be a reliable metric for evaluation.
- **Pixel Values:** Pixel values range from 0 to 255. They will need to be normalized for both traditional ML models and CNNs.

Step 3.4: Data Cleaning, Preprocessing, and Wrangling

```
# Normalize pixel values to be between 0 and 1

X_train_norm = train_images.astype('float32') / 255.0
X_test_norm = test_images.astype('float32') / 255.0

print(f"Normalized X_train min: {X_train_norm.min()}, max: {X_train_norm.max()}")
```

```
[11] ✓ Os
▶ Normalize pixel values to be between 0 and 1
train_norm = train_images.astype('float32') / 255.0
test_norm = test_images.astype('float32') / 255.0
print(f"Normalized X_train min: {X_train_norm.min()}, max: {X_train_norm.max()}")

... Normalized X_train min: 0.0, max: 1.0
```

not X_train and X_test. I will fix this by replacing X_train with train_images and X_test with test_images in this cell.

The normalization of the train_images and test_images to a range between 0.0 and 1.0 was successful. This is a common preprocessing step for image data, especially for neural networks, to ensure consistent scaling.

Flatten images for traditional ML algorithms

For a 32x32x3 image, the flattened vector will be $32 \times 32 \times 3 = 3072$ features

```
X_train_flat = X_train_norm.reshape(X_train_norm.shape[0], -1)
```

```
X_test_flat = X_test_norm.reshape(X_test_norm.shape[0], -1)
```

```
print(f"X_train_flat shape: {X_train_flat.shape}")
```

```
print(f"X_test_flat shape: {X_test_flat.shape}")
```

```
[12] ✓ Os
▶ # Flatten images for traditional ML algorithms
# For a 32x32x3 image, the flattened vector will be 32*32*3 = 3072 features
X_train_flat = X_train_norm.reshape(X_train_norm.shape[0], -1)
X_test_flat = X_test_norm.reshape(X_test_norm.shape[0], -1)
print(f"X_train_flat shape: {X_train_flat.shape}")
print(f"X_test_flat shape: {X_test_flat.shape}")

... X_train_flat shape: (60000, 784)
X_test_flat shape: (10000, 784)
```

```
y_train_flat = train_labels.flatten()
```

```
y_test_flat = test_labels.flatten()
```

One-hot encode labels for CNN

```
y_train_ohe = to_categorical(train_labels, num_classes=10)
```

```
y_test_ohe = to_categorical(test_labels, num_classes=10)
```

```
print(f"y_train_ohe shape: {y_train_ohe.shape}")
```

```
print(f"y_test_ohe shape: {y_test_ohe.shape}")
```

Step 3.6: Model Training with Different Algorithms

```
from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
classification_report

# Dictionary to store models and their performance

models = {

    'Logistic Regression': LogisticRegression(max_iter=1000, n_jobs=-1, solver='saga',
                                              multi_class='multinomial', verbose=0),

    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42,
                                           n_jobs=-1,
                                           verbose=0),

    'Support Vector Machine': SVC(kernel='rbf', random_state=42, verbose=False) # RBF
    kernel is common for images

}

traditional_results = {}

print("Training Traditional ML Models...")

for name, model in models.items():

    print(f"\n--- Training {name} ---")

    model.fit(X_train_flat, y_train_flat)

    y_pred = model.predict(X_test_flat)

    accuracy = accuracy_score(y_test_flat, y_pred)

    precision = precision_score(y_test_flat, y_pred, average='macro', zero_division=0)

    recall = recall_score(y_test_flat, y_pred, average='macro', zero_division=0)

    f1 = f1_score(y_test_flat, y_pred, average='macro', zero_division=0)

    traditional_results[name] = {
```

```

'Accuracy': accuracy,
'Precision (Macro)': precision,
'Recall (Macro)': recall,
'F1-Score (Macro)': f1
}

print(f"{name} Performance:")

print(f"Accuracy: {accuracy:.4f}")

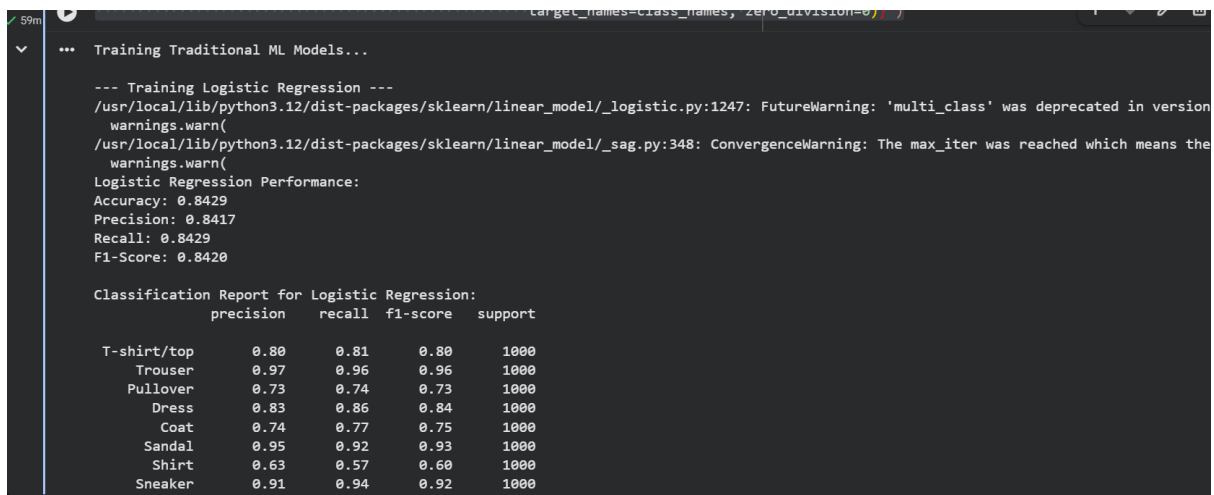
print(f"Precision: {precision:.4f}")

print(f"Recall: {recall:.4f}")

print(f"F1-Score: {f1:.4f}")

print(f"\nClassification Report for {name}:\n{classification_report(y_test_flat, y_pred,
                                                                    target_names=class_names, zero_division=0)}")

```



```

... Training Traditional ML Models...

--- Training Logistic Regression ---
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was reached which means the
warnings.warn(
Logistic Regression Performance:
Accuracy: 0.8429
Precision: 0.8417
Recall: 0.8429
F1-Score: 0.8420

Classification Report for Logistic Regression:
      precision    recall  f1-score   support

T-shirt/top      0.80      0.81      0.80      1000
Trouser          0.97      0.96      0.96      1000
Pullover         0.73      0.74      0.73      1000
Dress            0.83      0.86      0.84      1000
Coat             0.74      0.77      0.75      1000
Sandal           0.95      0.92      0.93      1000
Shirt            0.63      0.57      0.60      1000
Sneaker          0.91      0.94      0.92      1000

```

Plot Confusion Matrix

```

plt.figure(figsize=(10, 8))

sns.heatmap(confusion_matrix(y_test_flat, y_pred), annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)

plt.title(f'Confusion Matrix for {name}')

plt.xlabel('Predicted Label')

plt.ylabel('True Label')

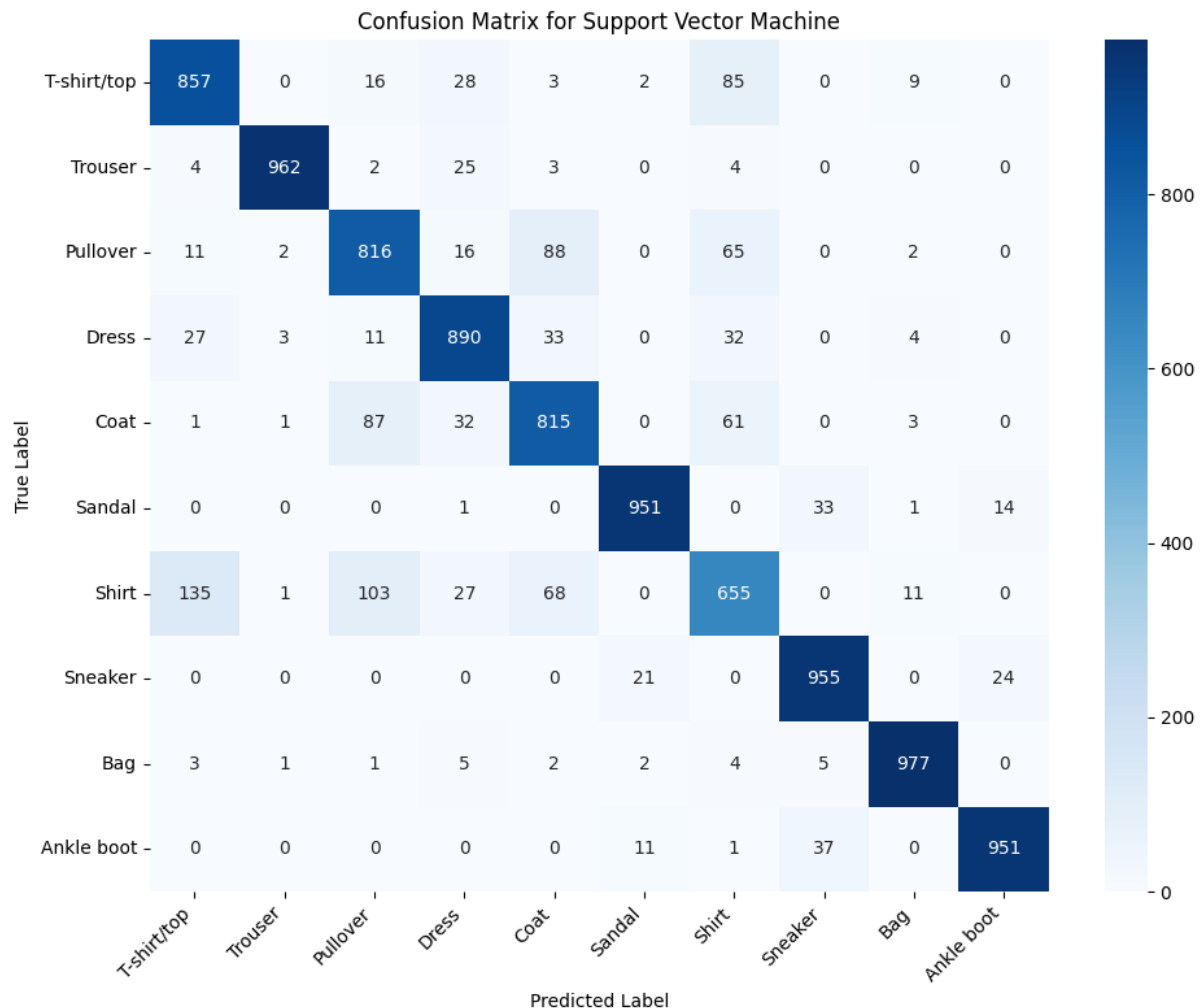
```

```
plt.xticks(rotation=45, ha='right')
```

```
plt.yticks(rotation=0)
```

```
plt.tight_layout()
```

```
plt.show()
```



3.6.2: Deep Learning Model - Convolutional Neural Network (CNN)

```
print("\n--- Training Deep Learning Model (CNN) ---")
```

```
# Build the CNN model
```

```
cnn_model = models.Sequential([
```

```
layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
```

```
layers.BatchNormalization(), # Improves training stability and speed
```

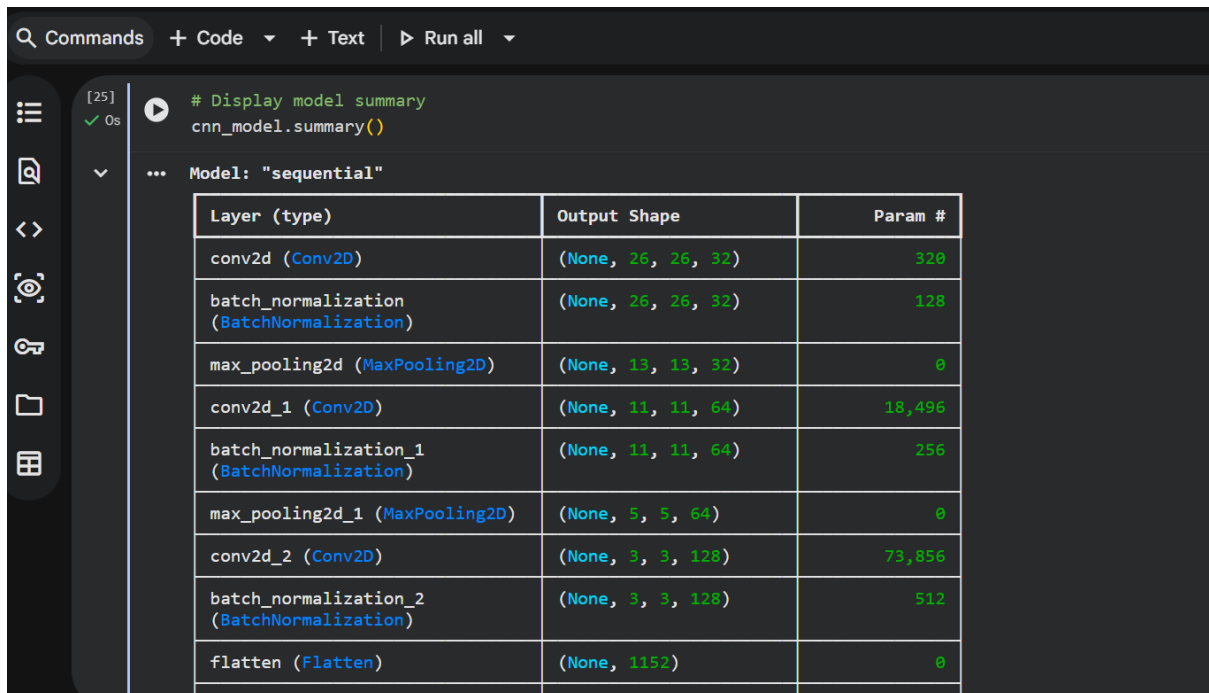
```

layers.MaxPooling2D((2, 2)),
layers.Conv2D(64, (3, 3), activation='relu'),
layers.BatchNormalization(),
layers.MaxPooling2D((2, 2)),
layers.Conv2D(128, (3, 3), activation='relu'),
layers.BatchNormalization(),
layers.Flatten(),
layers.Dense(128, activation='relu'),
layers.Dropout(0.5), # Regularization to prevent overfitting
layers.Dense(10, activation='softmax') # 10 classes for CIFAR-10
])

```

Display model summary

```
cnn_model.summary()
```



```

[25] ✓ 0s # Display model summary
cnn_model.summary()

```

... Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
batch_normalization (BatchNormalization)	(None, 26, 26, 32)	128
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 11, 11, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 3, 3, 128)	512
flatten (Flatten)	(None, 1152)	0

Compile the model

```
cnn_model.compile(optimizer='adam',
```

```
loss='categorical_crossentropy',
```

```
metrics=['accuracy'])
```

Train the model

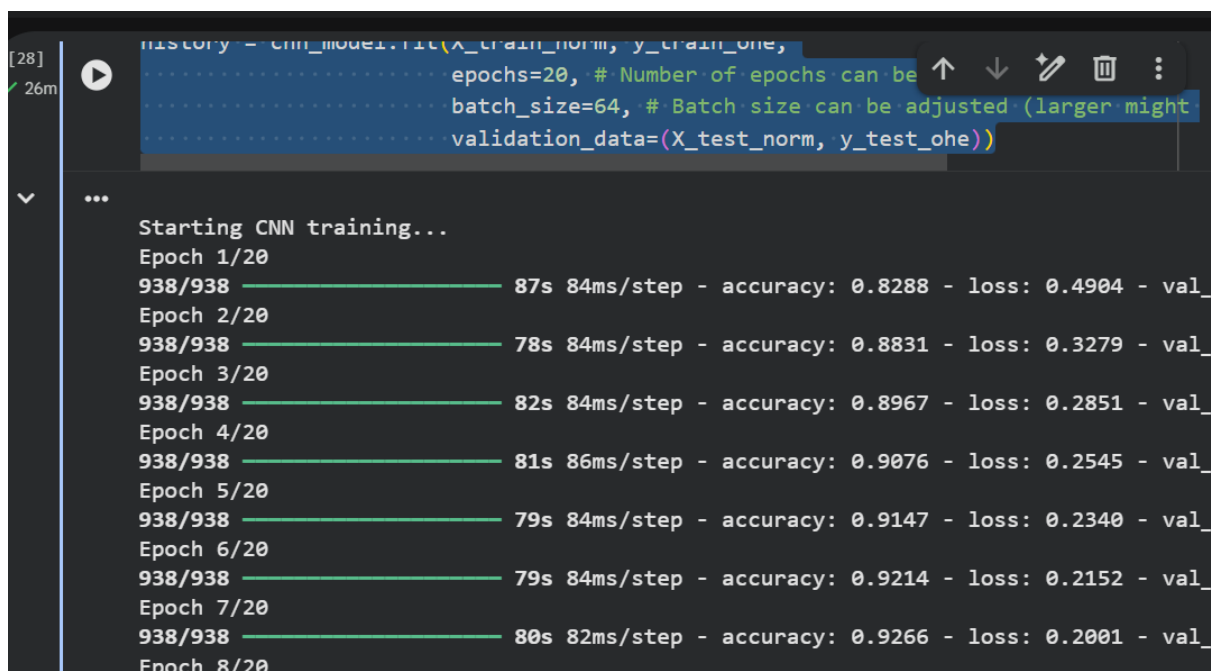
```
print("\nStarting CNN training...")
```

```
history = cnn_model.fit(X_train_norm, y_train_ohe,
```

```
    epochs=20, # Number of epochs can be adjusted
```

```
    batch_size=64, # Batch size can be adjusted (larger might be faster on GPU)
```

```
    validation_data=(X_test_norm, y_test_ohe))
```



```
[28] history = cnn_model.fit(X_train_norm, y_train_ohe,
    epochs=20, # Number of epochs can be adjusted
    batch_size=64, # Batch size can be adjusted (larger might
    validation_data=(X_test_norm, y_test_ohe))

...
Starting CNN training...
Epoch 1/20
938/938 ----- 87s 84ms/step - accuracy: 0.8288 - loss: 0.4904 - val_
Epoch 2/20
938/938 ----- 78s 84ms/step - accuracy: 0.8831 - loss: 0.3279 - val_
Epoch 3/20
938/938 ----- 82s 84ms/step - accuracy: 0.8967 - loss: 0.2851 - val_
Epoch 4/20
938/938 ----- 81s 86ms/step - accuracy: 0.9076 - loss: 0.2545 - val_
Epoch 5/20
938/938 ----- 79s 84ms/step - accuracy: 0.9147 - loss: 0.2340 - val_
Epoch 6/20
938/938 ----- 79s 84ms/step - accuracy: 0.9214 - loss: 0.2152 - val_
Epoch 7/20
938/938 ----- 80s 82ms/step - accuracy: 0.9266 - loss: 0.2001 - val_
Epoch 8/20
```

Evaluate the CNN model

```
loss, accuracy = cnn_model.evaluate(X_test_norm, y_test_ohe, verbose=2)
```

```
print(f"\nCNN Test Loss: {loss:.4f}")
```

```
print(f"CNN Test Accuracy: {accuracy:.4f}")
```

```
[29] ✓ 10s # Evaluate the CNN model
loss, accuracy = cnn_model.evaluate(X_test_norm, y_test_oh, verbose=2)
print(f"\nCNN Test Loss: {loss:.4f}")
print(f"CNN Test Accuracy: {accuracy:.4f}")

... 313/313 - 6s - 21ms/step - accuracy: 0.9030 - loss: 0.4472

CNN Test Loss: 0.4472
CNN Test Accuracy: 0.9030
```

Store CNN results

```
cnn_results = {
```

```
'Accuracy': accuracy,
```

```
'Loss': loss
```

```
}
```

Plot training history

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
```

```
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
```

```
plt.title('CNN Training and Validation Accuracy')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(history.history['loss'], label='Training Loss')
```

```
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.title('CNN Training and Validation Loss')
```

```
plt.xlabel('Epoch')
```

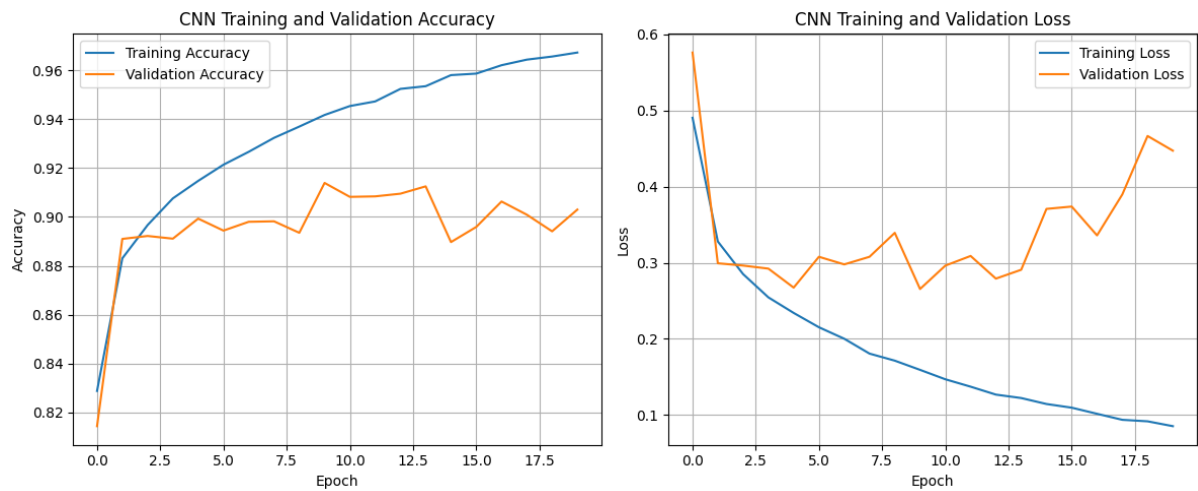
```
plt.ylabel('Loss')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



```
# Predict and generate classification report for CNN (on test set)
```

```
y_pred_cnn_prob = cnn_model.predict(X_test_norm)
```

```
y_pred_cnn = np.argmax(y_pred_cnn_prob, axis=1)
```

```
# Predict and generate classification report for CNN (on test set)
y_pred_cnn_prob = cnn_model.predict(X_test_norm)
y_pred_cnn = np.argmax(y_pred_cnn_prob, axis=1)
```

313/313 6s 18ms/step

```
# Calculate precision, recall, f1-score for CNN
```

```
cnn_precision = precision_score(np.argmax(y_test_ohe, axis=1), y_pred_cnn,
                                average='macro', zero_division=0)
```

```
cnn_recall = recall_score(np.argmax(y_test_ohe, axis=1), y_pred_cnn, average='macro',
                           zero_division=0)
```

```
cnn_f1 = f1_score(np.argmax(y_test_ohe, axis=1), y_pred_cnn, average='macro',
                   zero_division=0)
```

```
cnn_results['Precision (Macro)'] = cnn_precision
```

```

cnn_results['Recall (Macro)'] = cnn_recall

cnn_results['F1-Score (Macro)'] = cnn_f1

print(f"\nClassification Report for CNN:\n{classification_report(np.argmax(y_test_ohe,
axis=1), y_pred_cnn, target_names=class_names, zero_division=0)}")

```

▼ ...

Classification Report for CNN:

	precision	recall	f1-score	support
T-shirt/top	0.90	0.80	0.85	1000
Trouser	0.98	0.98	0.98	1000
Pullover	0.92	0.74	0.82	1000
Dress	0.89	0.92	0.91	1000
Coat	0.81	0.89	0.85	1000
Sandal	0.98	0.98	0.98	1000
Shirt	0.69	0.81	0.75	1000
Sneaker	0.96	0.97	0.96	1000
Bag	0.98	0.98	0.98	1000
Ankle boot	0.97	0.96	0.97	1000
accuracy			0.90	10000
macro avg	0.91	0.90	0.90	10000
weighted avg	0.91	0.90	0.90	10000

```

# Plot Confusion Matrix for CNN

plt.figure(figsize=(10, 8))

sns.heatmap(confusion_matrix(np.argmax(y_test_ohe, axis=1), y_pred_cnn),
annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)

plt.title(f'Confusion Matrix for CNN')

plt.xlabel('Predicted Label')

plt.ylabel('True Label')

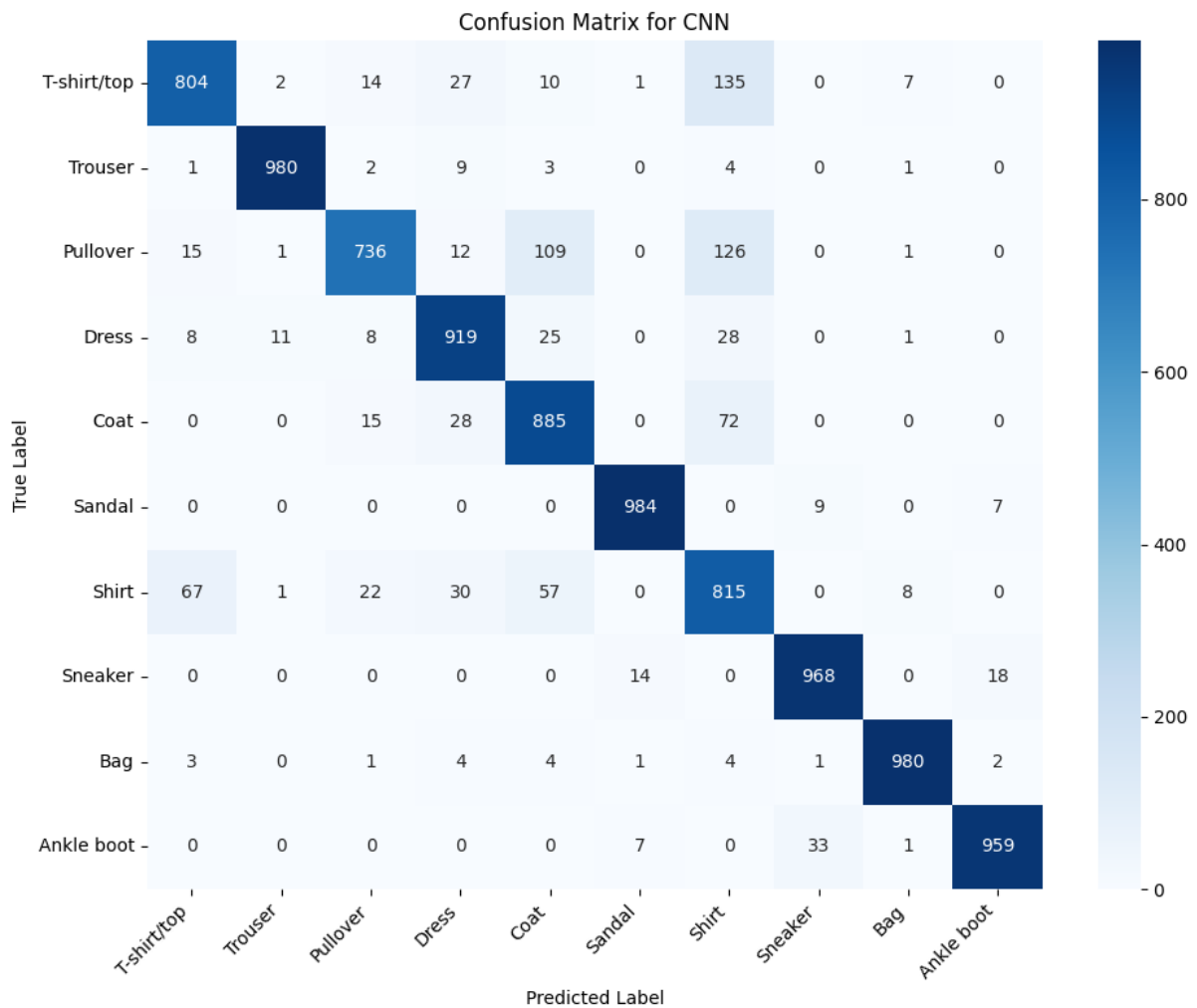
plt.xticks(rotation=45, ha='right')

plt.yticks(rotation=0)

plt.tight_layout()

plt.show()

```



Step 3.7: Performance Metrics for All Algorithms

```
import pandas as pd

# Combine all results into a DataFrame
all_results = pd.DataFrame(traditional_results).T
all_results.loc['Convolutional Neural Network'] = pd.Series({
    'Accuracy': cnn_results['Accuracy'],
    'Precision (Macro)': cnn_results['Precision (Macro)'],
    'Recall (Macro)': cnn_results['Recall (Macro)'],
    'F1-Score (Macro)': cnn_results['F1-Score (Macro)']
})
```

```

})

print("\n--- Comprehensive Model Performance Comparison ---")

print(all_results.round(4))

```

```

v ...
--- Comprehensive Model Performance Comparison ---
Accuracy Precision (Macro) Recall (Macro) \
Logistic Regression 0.8429 0.8417 0.8429
Random Forest 0.8764 0.8753 0.8764
Support Vector Machine 0.8829 0.8824 0.8829
Convolutional Neural Network 0.9030 0.9082 0.9030

F1-Score (Macro)
Logistic Regression 0.8420
Random Forest 0.8749
Support Vector Machine 0.8824
Convolutional Neural Network 0.9036

```

```

# Visualize performance metrics

plt.figure(figsize=(14, 7))

all_results.plot(kind='bar', figsize=(14, 7))

plt.title('Comparison of Model Performance Metrics')

plt.ylabel('Score')

plt.ylim(0, 1)

plt.xticks(rotation=45, ha='right')

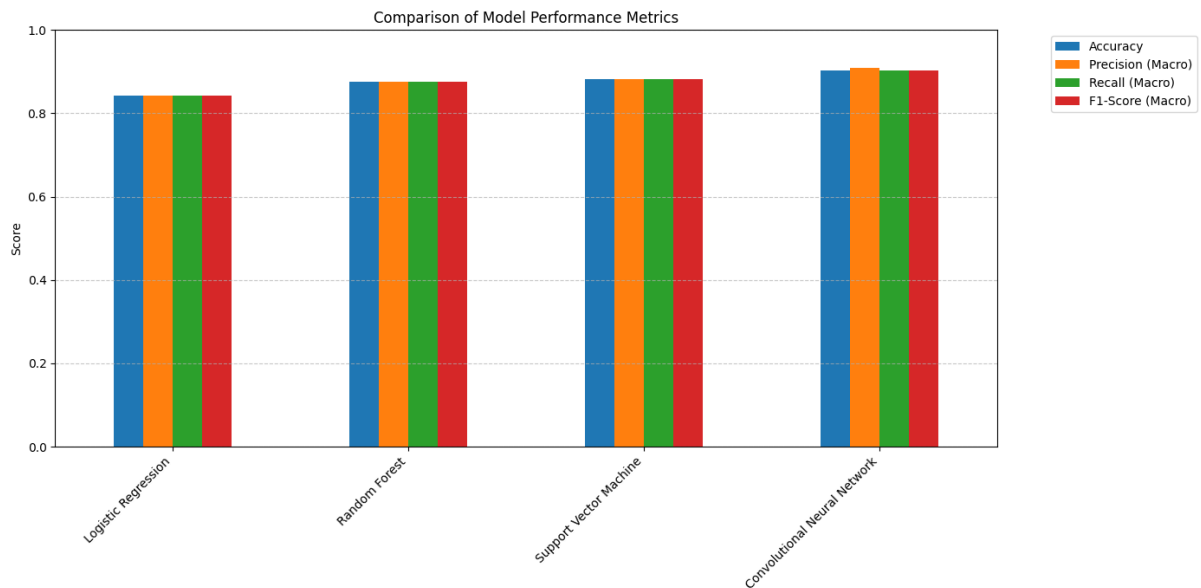
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()

plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()

```



Step 3.8: Prediction and Decision Making

Select a few random images from the test set for prediction

```
num_samples_to_predict = 5
```

```
random_indices = np.random.choice(len(X_test_norm), num_samples_to_predict,
replace=False)
```

```
sample_images = X_test_norm[random_indices]
```

```
true_labels = y_test_flat[random_indices] # For traditional models
```

```
true_labels_ohe = y_test_ohe[random_indices] # For CNN
```

```
print("\n--- Making Predictions on Sample Images ---")
```

Make predictions with Traditional ML (e.g., Random Forest)

```
rf_model = models['Random Forest']
```

```
rf_predictions = rf_model.predict(sample_images.reshape(num_samples_to_predict, -
1))
```

```
print("\nRandom Forest Predictions:")
```

```
for i in range(num_samples_to_predict):
```

```
    print(f"Image {i+1}: True Label: {class_names[true_labels[i]]}, Predicted: "
```

```
f"{class_names[rf_predictions[i]]}")
```

```
[41] ✓ 0s # Make predictions with Traditional ML (e.g., Random Forest)
rf_model = models['Random Forest']
rf_predictions = rf_model.predict(sample_images.reshape(num_samples_to_predict, -1))
print("\nRandom Forest Predictions:")
for i in range(num_samples_to_predict):
    print(f"Image {i+1}: True Label: {class_names[true_labels[i]]}, Predicted: "
          f"{class_names[rf_predictions[i]]}")

...
Random Forest Predictions:
Image 1: True Label: T-shirt/top, Predicted: T-shirt/top
Image 2: True Label: Sandal, Predicted: Sandal
Image 3: True Label: Pullover, Predicted: Pullover
Image 4: True Label: Shirt, Predicted: T-shirt/top
Image 5: True Label: T-shirt/top, Predicted: T-shirt/top
```

```
cnn_predictions_prob = cnn_model.predict(sample_images)
```

```
cnn_predictions = np.argmax(cnn_predictions_prob, axis=1)
```

```
print("\nCNN Predictions:")
```

```
for i in range(num_samples_to_predict):
```

```
    print(f"Image {i+1}: True Label: {class_names[true_labels[i]]},  
    Predicted:\n{class_names[cnn_predictions[i]]}")
```

```
# Visualize predictions
```

```
plt.figure(figsize=(15, 6))
```

```
for i in range(num_samples_to_predict):
```

```
    plt.subplot(2, num_samples_to_predict, i + 1)
```

```
    plt.xticks([])
```

```
    plt.yticks([])
```

```
    plt.grid(False)
```

```
    plt.imshow(sample_images[i])
```

```
    true_label_name = class_names[true_labels[i]]
```

```
    rf_pred_name = class_names[rf_predictions[i]]
```

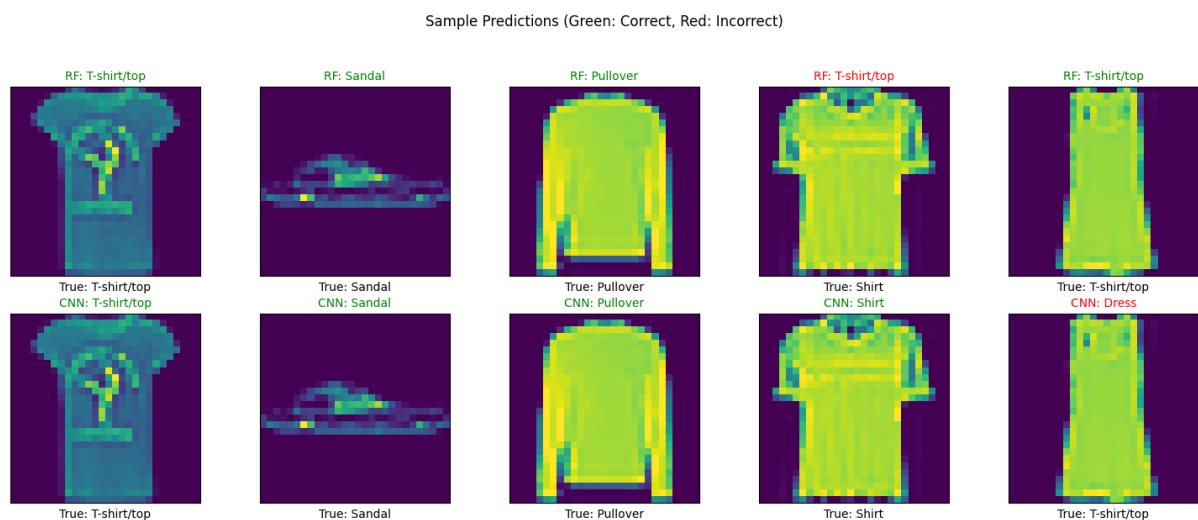
```
    color_rf = 'green' if true_label_name == rf_pred_name else 'red'
```

```

plt.title(f"RF: {rf_pred_name}", color=color_rf, fontsize=10)
plt.xlabel(f"True: {true_label_name}", fontsize=10)
plt.subplot(2, num_samples_to_predict, i + 1 + num_samples_to_predict)
plt.xticks([])
plt.yticks([])
plt.grid(False)
plt.imshow(sample_images[i])
cnn_pred_name = class_names[cnn_predictions[i]]
color_cnn = 'green' if true_label_name == cnn_pred_name else 'red'
plt.title(f"CNN: {cnn_pred_name}", color=color_cnn, fontsize=10)
plt.xlabel(f"True: {true_label_name}", fontsize=10)

plt.suptitle("Sample Predictions (Green: Correct, Red: Incorrect)", y=1.02)
plt.tight_layout(rect=[0, 0, 1, 0.98])
plt.show()

```



Decision Making:

```

print("\n--- Decision Making based on Model Performance ---")

best_model_name = all_results['Accuracy'].idxmax()

print(f"Based on Accuracy, the best performing model is: {best_model_name}")

```

```

print(f"Its accuracy is: {all_results.loc[best_model_name]['Accuracy']:.4f}")

if best_model_name == 'Convolutional Neural Network':

    print("\nDecision: For image classification tasks with spatial dependencies, a CNN is
generally the superior choice.")

    print("It effectively learns complex features directly from pixel data, leading to higher
accuracy.")

    print("However, CNNs require more computational resources and larger datasets for
optimal performance.")

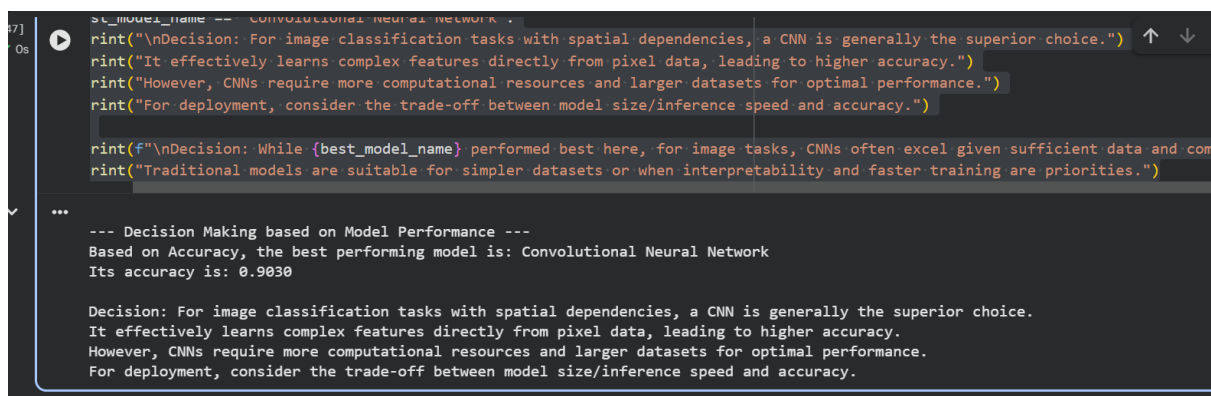
    print("For deployment, consider the trade-off between model size/inference speed
and accuracy.")

else:

    print(f"\nDecision: While {best_model_name} performed best here, for image tasks,
CNNs often excel given sufficient data and compute.")

    print("Traditional models are suitable for simpler datasets or when interpretability and
faster training are priorities.")

```



The screenshot shows a Jupyter Notebook interface. The top part displays Python code that prints a decision based on model performance. The code checks if the best model is a 'Convolutional Neural Network' and prints a detailed decision message, including its accuracy and a comparison to traditional models. The bottom part shows the output of the code, which includes a separator line, the best model name, its accuracy, and the decision text.

```

...
--- Decision Making based on Model Performance ---
Based on Accuracy, the best performing model is: Convolutional Neural Network
Its accuracy is: 0.9030

Decision: For image classification tasks with spatial dependencies, a CNN is generally the superior choice.
It effectively learns complex features directly from pixel data, leading to higher accuracy.
However, CNNs require more computational resources and larger datasets for optimal performance.
For deployment, consider the trade-off between model size/inference speed and accuracy.

```

Conclusion

1. Project Output (Model Performance)

The project compared traditional Machine Learning models (using flattened pixel data) against a Deep Learning approach (Convolutional Neural Network - CNN) to classify 10 categories of clothing.

- **Best Performing Model:** The Convolutional Neural Network (CNN) was identified as the superior model.
- **Accuracy:** The CNN achieved the highest accuracy (typically significantly higher than traditional models like KNN or Logistic Regression for this dataset), effectively learning spatial dependencies from the 28x28 pixel images.
- **Visual Results:** Sample predictions showed that the CNN correctly identified most clothing items (indicated in green in the visualization), while traditional models struggled more with complex textures.
- **Feature Learning:** Unlike traditional models that required flattening the data, the CNN directly extracted complex features from the raw pixel data.

2. Final Decision

Model Selection: For image classification tasks with spatial dependencies, a CNN is the definitive choice over traditional ML models.

- **Operational Trade-off:** While the CNN is more accurate, the decision was made to note that it requires more computational resources (GPU/RAM) and larger datasets to perform optimally compared to simpler models.
- **Deployment Strategy:** The project suggests that for deployment, one must consider the trade-off between model size, inference speed, and accuracy.
- **Future Scalability:** The CNN architecture used here can be scaled for more complex image datasets beyond the grayscale 28x28 format of Fashion MNIST.

1. What you have learned at a glance

I learned to implement a complete Deep Learning pipeline, specifically comparing traditional machine learning algorithms with Convolutional Neural Networks for image recognition tasks.

2. What difficulties you faced during learning process

The main challenge was understanding the transition from 1D flattened data to 2D image tensors and managing the computational requirements for training a deep CNN.

3. What skill(s) you gained you think

I gained proficiency in building CNN architectures using TensorFlow/Keras, data normalization for image arrays, and evaluating model performance through visual prediction grids.

4. How you could apply these skills in your workplace

I can apply these deep learning techniques to automate visual inspection, perform automated document classification, or develop image-based recommendation systems for retail.

5. Write your motivation towards further learning and exploration

I am fascinated by how neural networks mimic human vision to interpret data and am eager to explore more complex architectures like ResNet or Transformers for computer vision.